

even — P2P payments (full stack)

A working Venmo-style app: real accounts, a Postgres wallet ledger, and Stripe Connect for funding and cash-outs. The peer-to-peer transfer is an atomic move in your own ledger; Stripe handles money in (card) and money out (bank). Money routes are idempotent and rate-limited.

```
even/
├─ server/    Node + Express + Prisma/Postgres — auth, ledger, Stripe, webhooks
└─ web/       Vite + React client — auth screen + wallet UI
```

Quick start (one command)

```
unzip even-app.zip && cd even
make dev
```

`make dev` starts Postgres, applies migrations, installs deps on first run, launches the API (:4000) and web app (:5173), and — if the Stripe CLI is installed and logged in — starts the webhook listener with its signing secret wired in automatically. Ctrl-C stops everything; `make stop` also shuts down the database container.

Then open <http://localhost:5173>, register two accounts (use a normal window and an incognito one), **Add funds** on one with Stripe test card 4242 4242 4242 4242 , and **Pay** the other by @handle.

Other targets: `make test` (unit), `make test-int` (ledger vs. real Postgres), `make stop`.

Run it locally

1. Postgres — one command from the repo root:

```
docker compose up -d db
```

2. Backend

```
cd server
```

```

npm install                # runs `prisma generate`
cp .env.example .env      # set DATABASE_URL, JWT_SECRET, Stripe keys
npm run migrate            # applies the committed migration (prisma migrate de
npm run dev                # http://localhost:4000

```

3. Stripe keys (test mode) — <https://dashboard.stripe.com/test/apikeys>

- STRIPE_SECRET_KEY → .env
- Forward webhooks so top-ups credit wallets:

```

stripe login
stripe listen --forward-to localhost:4000/api/stripe/webhook

```

Put the `whsec_...` into `STRIPE_WEBHOOK_SECRET`.

- Generate `JWT_SECRET`:

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

4. Frontend

```

cd web
npm install
cp .env.example .env      # VITE_API_URL=http://localhost:4000
npm run dev                # http://localhost:5173

```

Register two accounts in two browser profiles, **Add funds** on one (Stripe test card 4242 4242 4242 4242, any future date/CVC), then **Pay** the other.

How the money moves

Action	What happens
Add funds	Stripe Checkout charges a card → webhook credits <code>balanceCents</code>
Pay	Atomic Prisma transaction between two users — no Stripe call
Request	Logs a pending request on the payer's feed
Cash out	Stripe Connect transfer + payout to the user's connected bank account

Balances are integer **cents** everywhere — never floats. The overdraft guard is a conditional

`updateMany (balanceCents >= amount)` inside a `$transaction`, so concurrent sends can't drive a balance negative.

Safety on money routes

- **Idempotency.** The client sends an `Idempotency-Key` header on pay, request, top-up, and cash-out. The server reserves that key (unique per user), runs the handler once, and replays the stored response on any retry — so a double-tap or network retry never moves money twice. Every Stripe write also carries its own idempotency key, so the charge/transfer/payout can't be duplicated either.
- **Rate limiting.** 120 req/min per IP across the API, 12/min on auth (slows brute force), 30/min on money routes. `trust proxy` is set so limits use the real client IP behind Railway's proxy.

Deploy on Railway (from GitHub)

Push this repo to GitHub, then in Railway:

1. **New Project → Deploy from GitHub repo.**
2. **Add a Postgres database** (New → Database → Postgres). Railway sets `DATABASE_URL` and exposes it to services in the project.
3. **API service** — root directory `server/`. Uses `server/railway.json` (Nixpacks, healthcheck on `/api/health`). The `start` script runs `prisma migrate deploy` (applies committed migrations) then boots the server. Env vars: `DATABASE_URL` (reference the Postgres plugin), `JWT_SECRET`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `WEB_ORIGIN` (the web URL).
4. **Web service** — root directory `web/`, uses `web/railway.json`. Set `VITE_API_URL` to the API's public URL (build-time var).
5. **Stripe webhook** — add an endpoint in the Stripe Dashboard pointing at `https://your-api.up.railway.app/api/stripe/webhook`; put its signing secret in `STRIPE_WEBHOOK_SECRET`.

Schema changes later

Migrations are committed under `server/prisma/migrations`. To change the schema: edit `schema.prisma`, run `npm run migrate` locally (creates a new migration), commit it, and push — Railway applies it on the next deploy via `migrate deploy`.

Tests

The risky logic (cents math, the overdraw guard, feed direction, idempotency decisions) is factored into `server/logic.js` and unit-tested — no database needed:

```
cd server
npm test           # 16 unit tests, runs in ~1s
```

Ledger behavior against a real database is covered by integration tests (atomic transfer, overdraw rejection leaving balances untouched, and a concurrency test firing 10 simultaneous full-balance transfers where exactly one wins and the balance never goes negative):

```
docker compose up -d db
cd server
DATABASE_URL="postgresql://postgres:pg@localhost:5432/even_test?schema=public" \
  npx prisma migrate deploy
DATABASE_URL="postgresql://postgres:pg@localhost:5432/even_test?schema=public" \
  npm run test:integration
```

The integration suite refuses to run unless `DATABASE_URL` names a *test* database, so it can't wipe your dev data.

Deploy on Replit (single service)

The API can serve the built web app, so the whole thing runs as one Replit service on one URL — which is exactly what makes pay-me links work.

1. **Import** this repo into Replit (Create Repl → Import from GitHub), or drag the unzipped folder in.
2. **Add a database:** open the **Database** tool and create a PostgreSQL database. Replit sets `DATABASE_URL` automatically.
3. **Add Secrets** (Tools → Secrets): `JWT_SECRET` (32+ chars), `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`. `WEB_ORIGIN` is optional — it defaults to your repl's domain.
4. **Run.** `.replit` runs `replit-run.sh`, which installs, builds the web app, applies migrations, and starts the server serving API + web on one origin.
5. **Stripe webhook:** in the Stripe Dashboard add an endpoint at `https://<your-repl-domain>/api/stripe/webhook` subscribed to `checkout.session.completed` and `payout.paid/failed/canceled`, and put its signing secret in `STRIPE_WEBHOOK_SECRET`.

(Link payments and top-ups both depend on receiving `checkout.session.completed`.)

Pay-me links

Create a link from the wallet ("Create a pay-me link") with an optional fixed amount and note. You get a URL and a QR code to share anywhere. Anyone opening `https://<your-repl-domain>/pay/<slug>` can pay by card through Stripe Checkout — no account required — and the money lands in your balance (minus the platform fee). Links can be deactivated via the API.

Charging a fee (revenue)

The app can take a fee from the sender on each payment. It's off by default; turn it on in `server/.env`:

```
PLATFORM_FEE_BPS=150           # 1.5%
PLATFORM_FEE_FLAT_CENTS=0      # optional flat add-on, in cents
# PLATFORM_FEE_CAP_CENTS=1000 # optional cap per payment, in cents
```

The recipient always receives the full amount; the sender pays amount + fee. The fee is credited to the `@even` platform account (seeded by migration `0003`), which holds real balance you can cash out to your own bank via the normal Connect payout flow.

`Transaction.feeCents` records each fee for reporting, and `GET /api/config` lets the client show the fee before sending.

A note on economics: a fee on friend-to-friend sends is the spot users resist most, and card-funded balances already cost you ~2.9% at Stripe — so a small send fee can be underwater. The proven place for a consumer fee is optional instant cash-out. The fee logic here is configurable so you can move or tune it.

Before this touches real money — read this

Holding customer balances and moving money between users is **regulated money transmission**. Two realistic paths:

1. **Build on a provider that carries the licensing** — Stripe Treasury (stored balances + payouts) or **Dwolla** (ACH-focused P2P). This code uses standard Stripe Connect; moving held balances to Treasury is the production step.
2. **Obtain money-transmitter licenses yourself** — state-by-state in the US, plus KYC/AML. Slow and expensive; almost everyone picks option 1.

Also non-negotiable:

- The **secret key lives only on the server**. Never ship `sk_...` to the client.
- Do real **KYC** before enabling cash-out (Stripe Connect onboarding covers much of it).
- Idempotency + rate limiting are wired; add fraud monitoring and dispute handling before real volume.

This project is a correct, runnable foundation — not a licensed money transmitter on its own.